

Assignment 5

ViT LoRA Fine-Tuning & Adversarial Attacks

Priyansh Saxena |

B22EE075

ML-DL-Ops | IIT Jodhpur |

WandB: wandb.ai/priyansh-saxena/Assignment-5 |

HuggingFace: huggingface.co/b22ee075/vit-s-lora-cifar100 |

Github: github.com/priyanshsaxena01/MLOps-Priyansh_Saxena-B22EE075/Assignment-5

Table of Contents

- 1. Introduction & Setup
- 2. Q1 — ViT-S Fine-Tuning on CIFAR-100
 - 2.1 Baseline: Classification Head Only (No LoRA)
 - 2.2 LoRA Experiments (Rank × Alpha Grid)
 - 2.3 Per-Experiment Epoch Tables
 - 2.4 Test Accuracy Summary
 - 2.5 Optuna Hyperparameter Search
 - 2.6 Analysis & Observations
- 3. Q2 — Adversarial Attacks using IBM ART
 - 3.1 ResNet-18 Baseline Training on CIFAR-10
 - 3.2 FGSM: From Scratch vs IBM ART
 - 3.3 Adversarial Detection (PGD & BIM)
 - 3.4 Analysis & Observations
- 4. Conclusion

1. Introduction & Setup

This report documents the complete experimental results for Assignment 5, which covers two major tasks: (Q1) fine-tuning a pre-trained Vision Transformer (ViT-Small) on CIFAR-100 using Low-Rank Adaptation (LoRA) via PEFT, and (Q2) adversarial robustness experiments using the IBM Adversarial Robustness Toolbox (ART) on CIFAR-10. All experiments were executed inside a Docker container on an NVIDIA RTX 2050 GPU (4 GB VRAM) using PyTorch with FP16 mixed precision.

Experimental Configuration

Parameter	Q1 (ViT-S / CIFAR-100)	Q2 (ResNet / CIFAR-10)
Model	ViT-Small (vit_small_patch16_224)	ResNet-18 / ResNet-34
Dataset	CIFAR-100 (50k train, 10k test)	CIFAR-10 (50k train, 10k test)
Epochs	10	50 (classifier), 30 (detectors)
Batch Size	32	128 (ResNet-18), 64 (ResNet-34)
Optimizer	AdamW (lr=1e-4, wd=0.01)	SGD (lr=0.1, momentum=0.9)
Scheduler	CosineAnnealingLR	CosineAnnealingLR
Mixed Prec.	FP16 (AMP)	FP16 (AMP)
Framework	PyTorch + PEFT + timm	PyTorch + ART

2. Q1 — ViT-S Fine-Tuning on CIFAR-100

The goal is to adapt a ViT-Small model pre-trained on ImageNet to the 100-class CIFAR-100 dataset. Two strategies are compared: (a) training only the classification head, and (b) applying LoRA (Low-Rank Adaptation) to the query, key, and value projection matrices in every transformer block while also keeping the classification head trainable.

2.1 Baseline — Classification Head Only (No LoRA)

In this configuration, all backbone parameters are frozen. Only the final linear classification head (100 output neurons) is trained, resulting in only 38,500 trainable parameters out of 21.7 M total.

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	2.1634	1.1942	54.73	71.74
2	0.9960	0.9532	74.67	74.24
3	0.8404	0.8723	77.25	75.64
4	0.7701	0.8339	78.50	76.44
5	0.7314	0.8064	79.28	77.52
6	0.7009	0.7929	80.14	77.72
7	0.6850	0.7835	80.45	77.98
8	0.6708	0.7777	80.77	78.08
9	0.6649	0.7769	81.06	78.10
10	0.6603	0.7762	81.09	78.16

Best validation accuracy: 78.16% | Test accuracy: 77.22% | Trainable parameters: 38,500

2.2 LoRA Experiments — Rank × Alpha Grid

LoRA is injected into the Q, K, V attention projection matrices of all 12 transformer blocks. Dropout is fixed at 0.1 throughout. Nine combinations of rank $\in \{2, 4, 8\}$ and alpha $\in \{2, 4, 8\}$ are evaluated.

Exp	Rank	Alpha	Dropout	Trainable Params	Best Val Acc (%)	Test Acc (%)
1	2	2	0.1	75,364	89.14	89.22
2	2	4	0.1	75,364	89.48	89.27
3	2	8	0.1	75,364	89.40	89.51
4	4	2	0.1	112,228	88.88	89.20
5	4	4	0.1	112,228	89.14	89.43
6	4	8	0.1	112,228	89.70	89.13
7	8	2	0.1	185,956	89.14	89.17
8	8	4	0.1	185,956	89.18	89.65
9	8	8	0.1	185,956	89.38	89.57

★ *Best single-run test accuracy: 89.65% achieved by Exp8 (Rank=8, Alpha=4).*

2.3 Per-Experiment Epoch Tables

Each LoRA experiment is trained for 10 epochs. The tables below show the complete training trajectory for all 9 LoRA configurations.

Experiment 1: Exp1_LoRA_r2_a2 [Rank=2, Alpha=2, Dropout=0.1]

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	1.5319	0.5826	66.58	84.26
2	0.4506	0.4511	86.80	86.42
3	0.3723	0.4030	88.53	87.64
4	0.3324	0.3846	89.59	88.30
5	0.3025	0.3747	90.60	88.52
6	0.2828	0.3625	91.20	88.96
7	0.2701	0.3633	91.58	88.78
8	0.2597	0.3557	91.90	89.12
9	0.2533	0.3556	92.13	89.08
10	0.2522	0.3544	92.01	89.14

Experiment 2: Exp2_LoRA_r2_a4 [Rank=2, Alpha=4, Dropout=0.1]

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	1.4822	0.5720	67.39	84.62
2	0.4401	0.4522	87.07	86.28
3	0.3610	0.4001	88.97	87.78
4	0.3218	0.3840	89.97	88.26
5	0.2940	0.3671	90.74	88.68
6	0.2752	0.3640	91.32	88.76
7	0.2588	0.3581	91.88	89.12
8	0.2469	0.3526	92.27	89.28
9	0.2416	0.3523	92.32	89.32
10	0.2382	0.3507	92.58	89.48

Experiment 3: Exp3_LoRA_r2_a8 [Rank=2, Alpha=8, Dropout=0.1]

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	1.4553	0.5626	67.94	85.04
2	0.4395	0.4382	87.02	87.14
3	0.3568	0.3957	88.90	88.12
4	0.3128	0.3781	90.30	88.94

5	0.2868	0.3664	91.15	88.80
6	0.2638	0.3602	91.68	88.98
7	0.2505	0.3591	92.14	89.14
8	0.2390	0.3546	92.46	89.32
9	0.2304	0.3526	92.89	89.34
10	0.2287	0.3516	92.79	89.40

Experiment 4: Exp4_LoRA_r4_a2 [Rank=4, Alpha=2, Dropout=0.1]

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	1.5511	0.5803	66.56	84.74
2	0.4554	0.4531	86.75	86.72
3	0.3709	0.4147	88.70	87.36
4	0.3299	0.3958	89.89	88.00
5	0.3024	0.3806	90.47	88.42
6	0.2784	0.3704	91.20	88.58
7	0.2668	0.3662	91.62	88.78
8	0.2540	0.3589	92.07	88.88
9	0.2516	0.3578	92.14	88.86
10	0.2475	0.3574	92.17	88.84

Experiment 5: Exp5_LoRA_r4_a4 [Rank=4, Alpha=4, Dropout=0.1]

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	1.5143	0.5527	66.72	84.82
2	0.4346	0.4384	87.08	86.94
3	0.3602	0.4005	89.02	87.62
4	0.3204	0.3866	89.98	88.06
5	0.2925	0.3681	90.76	88.26
6	0.2690	0.3632	91.52	88.54
7	0.2534	0.3588	92.15	88.82
8	0.2448	0.3533	92.38	88.84
9	0.2391	0.3506	92.51	88.94
10	0.2365	0.3502	92.65	89.14

Experiment 6: Exp6_LoRA_r4_a8 [Rank=4, Alpha=8, Dropout=0.1]

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	1.4263	0.5230	68.10	85.40
2	0.4287	0.4264	87.30	87.54
3	0.3548	0.3888	89.03	88.28
4	0.3122	0.3732	90.40	88.84
5	0.2823	0.3630	90.93	88.94
6	0.2608	0.3500	91.74	89.70
7	0.2468	0.3442	92.07	89.56
8	0.2323	0.3416	92.70	89.68
9	0.2253	0.3412	92.84	89.56
10	0.2236	0.3401	93.03	89.66

Experiment 7: Exp7_LoRA_r8_a2 [Rank=8, Alpha=2, Dropout=0.1]

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	1.5356	0.5635	66.84	84.24
2	0.4519	0.4412	86.72	87.04
3	0.3700	0.4080	88.61	87.64
4	0.3315	0.3850	89.72	88.08
5	0.2993	0.3740	90.70	88.46
6	0.2838	0.3681	91.14	88.62
7	0.2680	0.3601	91.62	88.88
8	0.2575	0.3591	91.85	88.98
9	0.2527	0.3559	92.03	89.06
10	0.2499	0.3550	92.21	89.14

Experiment 8: Exp8_LoRA_r8_a4 [Rank=8, Alpha=4, Dropout=0.1]

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	1.4623	0.5643	67.97	84.38
2	0.4394	0.4461	86.91	86.62
3	0.3610	0.4015	88.97	87.84
4	0.3178	0.3766	90.12	88.28

5	0.2864	0.3709	91.02	88.36
6	0.2672	0.3603	91.63	88.70
7	0.2576	0.3561	91.85	88.96
8	0.2446	0.3535	92.31	89.16
9	0.2368	0.3532	92.56	89.12
10	0.2346	0.3518	92.58	89.18

Experiment 9: Exp9_LoRA_r8_a8 [Rank=8, Alpha=8, Dropout=0.1]

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	1.4126	0.5350	68.65	84.98
2	0.4223	0.4258	87.55	87.08
3	0.3437	0.3800	89.52	88.08
4	0.3060	0.3651	90.40	88.62
5	0.2786	0.3623	91.21	88.34
6	0.2576	0.3516	91.69	88.58
7	0.2376	0.3466	92.57	89.06
8	0.2292	0.3400	92.74	89.38
9	0.2205	0.3404	93.11	89.12
10	0.2180	0.3394	93.07	89.14

2.4 Test Accuracy Summary

The table below summarises final test accuracy and parameter counts across all experiments (including the no-LoRA baseline).

LoRA	Rank	Alpha	Dropout	Test Acc (%)	Trainable Params
Without	-	-	0.1	77.22	38,500
With	2	2	0.1	89.22	75,364
With	2	4	0.1	89.27	75,364
With	2	8	0.1	89.51	75,364
With	4	2	0.1	89.20	112,228
With	4	4	0.1	89.43	112,228
With	4	8	0.1	89.13	112,228
With	8	2	0.1	89.17	185,956
With	8	4	0.1	89.65	185,956
With	8	8	0.1	89.57	185,956

Best test accuracy: 89.65% (Rank=8, Alpha=4, highlighted above). LoRA consistently yields +12 percentage points over the head-only baseline while training fewer than 1% of total parameters.

2.5 Optuna Hyperparameter Search

Optuna is used to search for the optimal LoRA hyperparameters (rank, alpha, dropout, learning rate) over 20 trials. The search space is: rank $\in \{2,4,8\}$, alpha $\in \{2,4,8\}$, dropout $\in [0.0, 0.5]$, lr $\in [1e-5, 1e-3]$ (log-uniform).

Parameter	Value
Best Rank	4
Best Alpha	8
Best Dropout	0.297
Best Val Accuracy	90.02%
Number of Trials	20

The Optuna best model (Rank=4, Alpha=8) achieves 90.02% validation accuracy — surpassing all fixed-grid experiments — and has been pushed to HuggingFace Hub as 'Optuna_Best_r4_a8'.

2.6 Analysis & Observations

- **LoRA vs Head-Only:** LoRA with even the smallest configuration ($r=2$, $\alpha=2$) achieves ~89% test accuracy vs 77% for head-only fine-tuning, a gain of ~12 pp. This confirms that adapting the attention weights is crucial for domain transfer.
- **Effect of Rank:** Higher rank generally provides more expressive capacity. Rank 8 achieves the best

single-run result (89.65%), but differences across ranks are small (<0.5 pp), suggesting diminishing returns beyond $r=4$.

- **Effect of Alpha:** Alpha controls the effective learning rate of LoRA updates (scaling $= \alpha/r$). Higher alpha generally improves performance for small ranks (e.g., $r=2, \alpha=8$ outperforms $r=2, \alpha=2$ by ~ 0.3 pp), but the effect saturates.
- **Trainable Parameters:** LoRA uses as few as 75,364 trainable parameters (0.35% of total 21.7M), demonstrating extreme parameter efficiency.
- **Gradient Norms:** LoRA weight gradients are logged per epoch via WandB. Gradient norms remain stable throughout training, indicating no vanishing/exploding gradient issues with the PEFT adapter.
- **Optuna Search:** Optuna's TPE sampler finds a better configuration ($r=4, \alpha=8, lr=6.9e-4$) than any fixed-grid experiment, confirming the value of automated HPO. The 90.02% val accuracy is 0.32 pp above the best grid result.
- **Class-wise Accuracy:** Detailed per-class histograms show that fine-grained classes (e.g., small animals, insects) remain challenging. LoRA improves all classes over the head-only baseline, especially visually distinct categories.

3. Q2 — Adversarial Attacks using IBM ART

This section covers adversarial robustness experiments on the CIFAR-10 dataset. A ResNet-18 classifier is trained from scratch on clean data, then subjected to FGSM attacks (implemented from scratch and using IBM ART). Additionally, ResNet-34 models are trained as binary adversarial detectors for PGD and BIM attacks.

3.1 ResNet-18 Baseline Training on CIFAR-10

A ResNet-18 model is trained from scratch for 50 epochs on clean CIFAR-10 images using SGD with momentum and CosineAnnealingLR scheduling. The target accuracy is $\geq 72\%$.

Parameter	Value
Architecture	ResNet-18 (non-pretrained)
Dataset	CIFAR-10 (50k train / 10k test)
Epochs	50
Batch Size	128
Optimizer	SGD (lr=0.1, momentum=0.9, wd=5e-4)
Scheduler	CosineAnnealingLR (T_max=50)
Final Test Acc	99.37% (clean) ✓ exceeds 72% target

3.2 FGSM Attack: From Scratch vs IBM ART

The Fast Gradient Sign Method (FGSM) generates adversarial examples by perturbing input images in the direction of the gradient of the loss with respect to the input: $x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x L(\theta, x, y))$. Two implementations are compared: a manual PyTorch implementation and the IBM ART FastGradientMethod wrapper. Experiments run for $\epsilon \in \{0.00, 0.01, 0.03, 0.05, 0.10, 0.20, 0.30\}$.

ϵ	Clean Acc (%)	FGSM Scratch (%)	FGSM ART (%)	Drop Scratch (pp)	Drop ART (pp)
0.000	99.37	99.37	99.37	0.00	0.00
0.010	99.37	35.58	38.83	63.79	60.54
0.030	99.37	23.27	25.05	76.10	74.32
0.050	99.37	18.28	19.78	81.09	79.59
0.100	99.37	12.29	12.53	87.08	86.84
0.200	99.37	10.04	10.36	89.33	89.01
0.300	99.37	10.00	10.32	89.37	89.05

Both implementations produce nearly identical adversarial examples. At $\epsilon=0.01$, accuracy collapses from 99.37% to ~35–39%, confirming the effectiveness of FGSM. IBM ART's implementation exhibits ~3 pp higher

accuracy at low ϵ (likely due to input normalisation differences), but converges to the same floor (~10%) at high ϵ .

Key Findings from FGSM Comparison

- **Sharp accuracy drop:** Even tiny perturbations ($\epsilon=0.01$) cause accuracy to drop by >60 pp, demonstrating the vulnerability of undefended neural networks.
- **Saturation at high ϵ :** For $\epsilon \geq 0.2$ the model essentially performs at random-chance level (~10%), meaning most predictions are wrong regardless of the true class.
- **ART vs Scratch parity:** Both methods yield near-identical adversarial images and accuracy curves, validating the scratch implementation. Minor differences arise from floating-point precision in the gradient computation pipeline.
- **Visual quality:** At low ϵ the adversarial perturbations are imperceptible to the human eye; at $\epsilon=0.3$ visible noise is present but the image is still recognisable

3.3 Adversarial Detection with ResNet-34

Two ResNet-34 binary classifiers are trained to distinguish clean from adversarial images. Detector-PGD is trained on images generated by the Projected Gradient Descent (PGD) attack, and Detector-BIM is trained on images generated by the Basic Iterative Method (BIM). Both are trained for 30 epochs on CIFAR-10. Cross-evaluation is performed to test generalisation across attack types.

Detector Model	Attack Used at Test	Detection Accuracy
PGD-trained (ResNet-34)	PGD	99.95%
PGD-trained (ResNet-34)	BIM	99.28%
BIM-trained (ResNet-34)	BIM	99.95%
BIM-trained (ResNet-34)	PGD	99.97%

All four evaluation scenarios exceed the 70% detection accuracy requirement by a wide margin, achieving near-perfect detection (>99%). The detectors generalise well across attack types (PGD-trained detector detects BIM at 99.28% and vice versa).

Detector Architecture & Training Details

Parameter	Value
Architecture	ResNet-34 (modified final layer: 2 output classes)
Training Data	50% clean + 50% adversarial (PGD or BIM via ART)
PGD params	$\epsilon=0.031$, step_size=0.007, max_iter=10
BIM params	$\epsilon=0.031$, step_size=0.007, max_iter=10
Loss	Binary CrossEntropy
Training Epochs	30
Min Target Acc	$\geq 70\%$ (achieved: >99%)

3.4 Analysis & Observations

- **FGSM devastates undefended models:** A 99.37% clean-accuracy ResNet-18 drops to 35% under $\epsilon=0.01$ FGSM, confirming that standard training provides no adversarial robustness. This motivates adversarial detection as a complementary defence.
- **ART accelerates research:** IBM ART's high-level API simplifies attack implementation and enables fair comparison by standardising attack parameters. The library wraps the model in a PyTorchClassifier and handles gradient computation transparently.
- **PGD and BIM are closely related:** BIM is essentially iterated FGSM (PGD without the random start). The two attacks produce statistically indistinguishable adversarial distributions on CIFAR-10, explaining why cross-evaluation detection accuracy remains near perfect (>99%).
- **Detection is highly learnable:** Adversarial images generated by gradient-based attacks

carry statistical signatures (high-frequency noise patterns aligned with the gradient direction) that ResNet-34 can detect with near-perfect accuracy after 30 epochs of supervised training.

- **WandB logging:** 10 sample images for each attack type (FGSM-scratch, FGSM-ART, PGD, BIM) are uploaded to WandB for visual inspection and reproducibility.

4. Conclusion

This assignment demonstrates two important frontiers in modern deep learning:

- **Parameter-Efficient Fine-Tuning (Q1):** LoRA with rank 8 and alpha 4 achieves 89.65% test accuracy on CIFAR-100 by training only ~186k parameters out of 21.9M (0.85%), compared to 77.22% when only the classification head is trained. Optuna automated HPO further improves this to 90.02% val accuracy with rank 4, alpha 8. The results confirm that injecting low-rank adapters into Q, K, V projections is a highly effective strategy for domain adaptation of large vision transformers.
- **Adversarial Robustness (Q2):** Gradient-based attacks (FGSM, PGD, BIM) are extremely effective against standard ResNet-18, collapsing accuracy from 99.37% to near random at $\epsilon \geq 0.2$. However, binary adversarial detectors (ResNet-34) trained on attack-specific data achieve near-perfect detection (>99%) and generalise across attack types, offering a practical detection-based defence strategy.

All code, weights, plots, and results are available at:

- **GitHub (Assignment-5 branch):** github.com/priyansh-saxena/Assignment-5
- **WandB:** wandb.ai/priyansh-saxena/Assignment-5
- **HuggingFace:** <https://huggingface.co/b22ee075/vit-s-lora-cifar100>